UNIVERSIDAD DE COSTA RICA SEDE
REGIONAL DE LIMÓN
BACH. INFORMÁTICA EMPRESARIAL
PROGRAMA CURSO: Desarrollo de Software IV
II Ciclo, 2026

PRUEBA PARCIAL 1 TIEMPO: 3 horas Valor 100pts / 20%

La Universidad de Costa Rica fomenta la probidad académica como base del ejercicio profesional ético. Por lo tanto, el desarrollo de esta prueba práctica debe ser un ejercicio individual, auténtico y resultado del esfuerzo intelectual propio del estudiante.

Sobre el plagio y la colaboración

- **Prohibición de copia:** Queda estrictamente prohibido compartir código, utilizar repositorios de compañeros (presentes o de semestres anteriores) o entregar soluciones idénticas o parcialmente similares. La detección de similitudes técnicas mediante herramientas de análisis de código resultará en la anulación automática de la evaluación para todas las partes involucradas.
- **Trabajo autónomo:** El sistema de capas y la estructura de Docker deben ser configurados por cada estudiante. La copia, total o parcial, de la lógica de negocio o de datos será considerada una falta grave.

Sobre el uso de IA

Está prohibido utilizar IA para generar el código base del CRUD, las consultas SQL, la configuración del docker-compose o la lógica de negocio o datos completa. Generar una solución completa mediante prompts constituye un fraude académico, ya que el estudiante no está demostrando la competencia que se busca evaluar.

Defensa de la Solución: El profesor podrá solicitar una explicación detallada de cómo se implementaron las validaciones de estado en el contenedor de negocios. La incapacidad de explicar la lógica desarrollada será motivo suficiente para anular la nota.

Consecuencias

Cualquier acción que contravenga lo estipulado en este apartado será sancionada conforme a lo establecido en el **Reglamento de Régimen Académico Estudiantil** de la Universidad de Costa Rica. La honestidad en su formación profesional es tan importante como su capacidad técnica; les insto a realizar un trabajo ético y consciente

1. Objetivo

Evaluar la capacidad del estudiantado en el diseño e implementación de una aplicación web tipo API REST robusta, estructurada bajo una arquitectura en capas (Negocio y Datos), con persistencia administrada mediante el ORM Eloquent y base de datos MySQL, todo esto completamente orquestado y desplegado a través de contenedores con Docker Compose.

2. Requerimientos Técnicos Arquitectura (Multicapa)

Desarrollar **un único recurso asignado**, implementando un CRUD completo (Crear, Leer, Actualizar, Eliminar) bajo las siguientes especificaciones técnicas obligatorias:

1. **Lenguaje y Framework:** PHP con **Slim Framework** para el manejo de rutas y peticiones HTTP.
2. **Arquitectura en Capas:**
    - **Capa de Negocio:** Controladores, validaciones, reglas de negocio y lógica de procesamiento de peticiones.
    - **Capa de Datos / Persistencia:** Modelos de Eloquent ORM, repositorios o servicios de acceso a datos que interactúan directamente con la base de datos MySQL.
3. **Contenedorización (Docker Compose):**

El sistema completo deberá levantarse con un único archivo docker-compose.yml que configure exactamente **tres contenedores**:

- Contenedor 1: Capa de Negocio (Servidor web PHP/Slim).
- Contenedor 2: Capa de Datos (Módulo intermedio o servicio de lógica de acceso a datos).
- Contenedor 3: Servidor de base de datos MySQL (con persistencia de volúmenes).

4. **Endpoints Esperados:** Para el recurso asignado, se deben exponer mínimo los siguientes endpoints RESTful:
    - GET /api/{recurso} (Listar todos los registros)
    - GET /api/{recurso}/{id} (Obtener un registro específico)
    - POST /api/{recurso} (Crear un nuevo registro)
    - PUT /api/{recurso}/{id} (Actualizar un registro existente)
    - DELETE /api/{recurso}/{id} (Eliminar un registro)

5. Asignación de Recursos (Casos de Estudio)

Para evitar la duplicidad y asegurar la individualidad de la prueba, se distribuyen **cinco recursos diferentes**. Cada estudiante implementará el recurso que se le indique de forma personalizada según el número asignado.

RECURSO 1: Sistema de Gestión de Taller Mecánico (ordenes_trabajo)
RECURSO 2: Sistema de Matrícula Académica (cursos_matricula)
RECURSO 3: Control de Inventarios de Bodega (inventario_productos)
RECURSO 4: Gestión de Biblioteca Central (biblioteca_libros)
RECURSO 5: Sistema de Gestión de Salón de Belleza (citas_salon)

5. Criterios de Evaluación

- **Correcta configuración de Docker:** Funcionamiento del comando docker-compose up y conectividad entre contenedores.
- **Arquitectura:** Separación efectiva entre la Capa de Negocios y la Capa de Datos (uso de inyección de dependencias o interfaces recomendada).
- **Persistencia:** Uso correcto de PDO dentro de la capa de datos.
- **Código Limpio:** Uso de buenas prácticas en PHP, manejo de errores y códigos de estado HTTP correctos (200, 201, 404, 204, 409).
- **CRUD Funcional:** Todos los endpoints deben responder según las especificaciones.
- **Validación:** Se deberá implementar la validación para al menos uno de los campos.

Consideraciones

- **Usar únicamente la tabla que se le asignó para implementar el CRUD, de lo contrario no se tomará en cuenta el examen**
- Toda ejecución se llevará a cabo consumiendo los servicios de la capa de negocio, el cual a su vez consumirá los servicios de la capa de datos
- En el archivo docker compose todo **servicio** deberá ser antecedido con las siglas de su nombre y dos apellidos (ejemplo: lwgdb, lwgnegocio, lwgdatos.
- En la entrega **NO** deberá aparecer ningún otro servicio no solicitado
- No se puede subir, ni corregir ni re-subir nada después del tiempo límite de entrega (si se tiene problemas subiendo el examen a mediación, deberá indicarlo antes del vencimiento del tiempo límite, luego de este tiempo no se aceptarán entregas por ningún medio)

Entregable

- **Código Fuente en mediación:** Estructura de directorios limpia respetando el patrón de capas, en un único archivo comprimido.
- **Código Fuente en Repositorio Git:** Estructura de directorios limpia respetando el patrón de capas (5 pts. extra).
- **Archivo docker-compose.yml Funcional:** Capaz de levantar los 3 contenedores requeridos con un simple comando (docker compose up --build).
- **Documentación y Pruebas:** Colección de ResClient o script de pruebas para validar los endpoints REST desarrollados.

Indicadores	Valor	Obt
La aplicación se ejecuta de forma correcta sin errores o inconvenientes	10	
Correcta configuración de los contenedores	8	
Se puede crear un nuevo registro de forma correcta	15	
Se puede modificar un registro existente de forma correcta	15	
Se puede eliminar un registro de forma correcta	5	
Se implemento la validación de negocio solicitada de forma correcta	5	
Se retorna el estado resultante de forma correcta según sea el caso	8	
Los servicios fueron nombrados de la forma solicitada	6	
El cliente REST consume los servicios de la capa de negocio	8	
La capa de negocio consume los servicios de la capa de datos	8	
La entrega se hace de la forma solicitada	2	
La entrega no contiene código no necesario o no solicitado	10	
Total puntos	100	
Total Porcentaje	20%	

"Los grandes esfuerzos traen grandes recompensas."